

Taller experimental: comparación de algoritmos bioinspirados y metaheurísticos

Algoritmo genético, ACO, PSO, ascenso de colinas y temple simulado

Modalidad: Trabajo en equipos de 2 o 3 estudiantes

Valor: 5,0 puntos

Herramientas: Python, NumPy, Matplotlib, pandas y NetworkX

Propósito

Diseñar un experimento reproducible que permita comparar cinco métodos de optimización sobre un mismo problema. La comparación deberá considerar no solo la mejor solución, sino también la eficiencia computacional, velocidad de convergencia, estabilidad, robustez y escalabilidad.

Al finalizar, el estudiante estará en capacidad de:

- implementar y adaptar diferentes metaheurísticas a una representación común;
- diseñar una comparación justa mediante presupuestos equivalentes de evaluación;
- analizar experimentalmente calidad, costo computacional y variabilidad;
- formular conclusiones sustentadas en evidencia estadística y gráfica.

Problema común: viajante de comercio

Se dispone de un conjunto de n ciudades, donde cada ciudad i tiene coordenadas (x_i, y_i) . El objetivo es encontrar una ruta cerrada que visite cada ciudad exactamente una vez y regrese al punto inicial, minimizando la distancia total:

$$f(\pi) = \sum_{k=1}^{n-1} d_{\pi_k, \pi_{k+1}} + d_{\pi_n, \pi_1}, \quad d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}, \quad (1)$$

donde π es una permutación de las ciudades. Todas las implementaciones deberán usar exactamente las mismas instancias, matriz de distancias y función objetivo.

Instancias mínimas: generar problemas de $n \in \{20, 50, 100\}$ ciudades, con coordenadas uniformes en $[0, 100] \times [0, 100]$. Para cada tamaño se usarán al menos cinco instancias diferentes, identificadas mediante semillas públicas.

Representación por algoritmo

Algoritmo	Representación	Operadores o actualización mínima
Algoritmo genético (GA)	Permutación de ciudades	Selección por torneo, cruce OX o PMX, mutación por intercambio o inversión y elitismo.
ACO	Ruta construida probabilísticamente	Feromonas, visibilidad $1/d_{ij}$, evaporación y depósito asociado a la calidad de las rutas.
PSO	Vector continuo de claves aleatorias	Actualización de velocidad/posición; la ruta se obtiene ordenando las claves del vector.
Ascenso de colinas (HC)	Permutación	Vecindad por intercambio o 2-opt; conservar el vecino si mejora la solución.
Temple simulado (SA)	Permutación	Misma vecindad de HC; aceptar empeoramientos con $P = \exp(-\Delta f/T)$ y reducir T .

Para controlar variables, HC y SA deberán compartir el mismo generador de soluciones vecinas. Todos los algoritmos deberán devolver la mejor solución histórica, no solamente la solución de la última iteración.

Protocolo para una comparación justa

Una iteración no representa el mismo trabajo en los cinco métodos. Por tanto, el criterio principal de parada será el número de **evaluaciones de la función objetivo** (FEs). Se propone:

- presupuesto: 10 000 FEs para $n = 20$, 30 000 para $n = 50$ y 60 000 para $n = 100$;
- 30 ejecuciones independientes por combinación de algoritmo, tamaño e instancia;
- las mismas semillas maestras para todos los algoritmos;
- registrar el mejor costo cada vez que se alcance 1%, 5%, 10%, 20%, 40%, 60%, 80% y 100% del presupuesto;
- ejecutar las pruebas en el mismo computador, sin procesos pesados en segundo plano;
- excluir de la medición de tiempo la generación de ciudades y la producción de gráficas.

Cada algoritmo puede requerir parámetros diferentes, pero estos deben fijarse antes del experimento final. Se permite una fase piloto con instancias separadas de las usadas para reportar resultados.

Métricas obligatorias

Sea f^* el mejor valor de referencia conocido para una instancia y f_r el resultado de la ejecución r . Si no se conoce el óptimo, se empleará como referencia el mejor valor encontrado por cualquiera de los algoritmos en todas las ejecuciones.

$$\text{Error relativo}_r(\%) = 100 \frac{f_r - f^*}{f^*}, \quad (2)$$

$$\text{Tasa de éxito}(\%) = 100 \frac{\#\{r : \text{error}_r \leq \varepsilon\}}{R}, \quad (3)$$

$$\text{Mejora}(\%) = 100 \frac{f_{\text{inicial}} - f_r}{f_{\text{inicial}}}. \quad (4)$$

Use $\varepsilon = 1\%$ y $R = 30$. Además, registre:

- mejor, media, mediana y desviación estándar del costo final;
- tiempo total y tiempo por cada 1.000 evaluaciones;

- memoria máxima aproximada, medida con `tracemalloc`;
- evaluación en la que se obtuvo la mejor solución;
- curva de convergencia: mejor costo histórico frente a FEs;
- escalabilidad al pasar de 20 a 50 y 100 ciudades.

Importante: una sola ejecución o solamente el mejor resultado no constituye evidencia suficiente sobre el rendimiento de un algoritmo estocástico.

Desarrollo del taller

1. Diseño e implementación

Implemente una interfaz común para los cinco algoritmos. Cada función debe recibir la matriz de distancias, presupuesto, semilla y parámetros; debe devolver la mejor ruta, el mejor costo, historial, número de evaluaciones, tiempo y memoria.

```
def optimizar(distancias, presupuesto, semilla, parametros):  
    return {  
        "mejor_ruta": ruta,  
        "mejor_costo": costo,  
        "historial": historial, # pares (evaluaciones, mejor_costo)  
        "evaluaciones": evaluaciones,  
        "tiempo_s": tiempo,  
        "memoria_mb": memoria  
    }
```

Documente la codificación y los operadores utilizados. Incluya pruebas que verifiquen que cada ruta contiene todas las ciudades una sola vez y que el costo reportado coincide con el recalculado.

2. Diseño experimental y reproducibilidad

Construya automáticamente las instancias y ejecuciones del protocolo. Justifique los parámetros seleccionados y explique cómo garantiza igualdad de presupuesto. Guarde cada ejecución en un archivo CSV con, como mínimo:

```
algoritmo, n, instancia, repeticion, semilla, costo, error, tiempo, memoria, FEs_mejor
```

Incluya un archivo de configuración o diccionario centralizado para que el experimento pueda repetirse sin modificar el código fuente.

3. Eficiencia computacional y escalabilidad

Compare el tiempo, la memoria y el número de evaluaciones necesario para alcanzar un error menor o igual al 1%. Construya:

- a) gráfica del tiempo promedio frente al número de ciudades;
- b) gráfica de memoria máxima promedio frente al número de ciudades;
- c) tabla de tasa de éxito y FEs necesarias para alcanzar el umbral;
- d) análisis de cuál algoritmo escala mejor y por qué.

No confunda rapidez computacional con calidad: un método puede terminar antes y producir rutas más largas.

4. Rendimiento, convergencia y estabilidad

Presente los siguientes resultados:

- curvas de convergencia promedio con banda de variabilidad;
- diagramas de caja del error relativo final;
- tabla con media, mediana, desviación estándar, mejor resultado y tasa de éxito;
- rutas obtenidas por los cinco métodos en una misma instancia;
- ranking por calidad, estabilidad y velocidad de convergencia.

Para las curvas agregadas, interpole o registre los resultados en los mismos puntos del presupuesto de FEs. No promedie curvas que correspondan a ejes de evaluación diferentes.

5. Análisis crítico y conclusiones

Responda con evidencia:

- ¿Qué algoritmo obtuvo las rutas de menor costo y cuál fue el más estable?
- ¿Cuál alcanzó buenas soluciones usando menos evaluaciones?
- ¿Cuál tuvo menor tiempo y consumo de memoria? ¿Coincide con el de mejor calidad?
- ¿Cómo cambió el ranking al aumentar el número de ciudades?
- ¿Qué efecto tienen el componente poblacional de GA, ACO y PSO y la búsqueda de trayectoria única de HC y SA?
- ¿Cuándo resulta útil aceptar empeoramientos, como ocurre en SA, o mantener diversidad, como en los métodos poblacionales?
- ¿Existe un ganador absoluto? Proponga qué algoritmo usaría en tres escenarios: respuesta rápida, solución de alta calidad y recursos de memoria limitados.

Incluya una amenaza a la validez del estudio y una mejora que aplicaría en un experimento posterior.

Análisis estadístico sugerido

Como mínimo, reporte mediana y rango intercuartílico. Para sustentar diferencias entre los cinco algoritmos, puede aplicar la prueba de Friedman sobre resultados emparejados por instancia y repetición. Si se detectan diferencias significativas ($\alpha = 0,05$), realice comparaciones post hoc con corrección de Holm. Reporte también un tamaño del efecto o, al menos, los rangos promedio.

No es correcto concluir que un algoritmo es superior únicamente porque su media sea ligeramente menor. Deben considerarse variabilidad, magnitud de la diferencia y consistencia entre instancias.

Productos de entrega

Entregue un archivo comprimido con:

- `src/`: implementación modular de los cinco algoritmos;
- `experimentos/`: script que ejecuta automáticamente todas las pruebas;
- `resultados/`: CSV originales y tablas resumidas;
- `figuras/`: gráficas con títulos, unidades, leyendas y pie explicativo;
- `informe.pdf`: máximo 8 páginas, incluyendo metodología, resultados y conclusiones;
- `README.md`: instalación, dependencias, ejecución, semillas y estructura del proyecto;
- `requirements.txt`: versiones de las bibliotecas utilizadas.

El comando principal sugerido es:

```
python ejecutar_experimentos.py --config configuracion.json
```

Pregunta de profundización opcional

Diseñe un algoritmo híbrido que combine dos de los métodos estudiados; por ejemplo, GA o ACO con mejora local 2-opt, o PSO con temple simulado. Mantenga el mismo presupuesto total de evaluaciones y determine si la hibridación ofrece una mejora estadísticamente relevante frente a sus componentes individuales.

Una comparación válida controla el problema, el presupuesto y las condiciones de ejecución; después estudia conjuntamente calidad, costo y variabilidad.